Data Visualization Using Matplotlib

Analyzing Air Quality Index (AQI) Trends in a City

Problem Statement: Analyze air-quality trends and pollutant patterns over time using Matplotlib.

Dataset: `City_Air_Quality.csv`

1. Import the Dataset

In [2]:
```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("City_Air_Quality.csv", encoding="cp1252", low_memory=False)
display(df.head())
```

	stn_code	sampling_date	state	location	agency	type	so2	no2	rspm	spm	location_monitoring_station	pm2_5	date
0	150	February - M021990	Andhra Pradesh	Hyderabad	NaN	Residential, Rural and other Areas	4.8	17.4	NaN	NaN	NaN	NaN	1990-02-01
1	151	February - M021990	Andhra Pradesh	Hyderabad	NaN	Industrial Area	3.1	7.0	NaN	NaN	NaN	NaN	1990-02-01
2	152	February - M021990	Andhra Pradesh	Hyderabad	NaN	Residential, Rural and other Areas	6.2	28.5	NaN	NaN	NaN	NaN	1990-02-01
3	150	March - M031990	Andhra Pradesh	Hyderabad	NaN	Residential, Rural and other Areas	6.3	14.7	NaN	NaN	NaN	NaN	1990-03-01
4	151	March - M031990	Andhra Pradesh	Hyderabad	NaN	Industrial Area	4.7	7.5	NaN	NaN	NaN	NaN	1990-03-01

2. Explore the Dataset

In [3]:
```
print("Dataset shape:", df.shape)
print("\nColumn names:")
print(df.columns.tolist())
print("\nData types and non-null counts:")
df.info()
print("\nMissing values:")
display(df.isnull().sum().to_frame("Missing Values"))
print("\nSummary statistics:")
display(df.describe(include="all").T)
```

```
Dataset shape: (435742, 13)

Column names:
['stn_code', 'sampling_date', 'state', 'location', 'agency', 'type', 'so2', 'no2', 'rspm', 'spm', 'location_monitoring_station', 'pm2_5', 'date']

Data types and non-null counts:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 435742 entries, 0 to 435741
Data columns (total 13 columns):
 #   Column                        Non-Null Count   Dtype
---  ------                        --------------   -----
 0   stn_code                      291665 non-null  object
 1   sampling_date                 435739 non-null  object
 2   state                         435742 non-null  object
 3   location                      435739 non-null  object
 4   agency                        286261 non-null  object
 5   type                          430349 non-null  object
 6   so2                           401096 non-null  float64
 7   no2                           419509 non-null  float64
 8   rspm                          395520 non-null  float64
 9   spm                           198355 non-null  float64
 10  location_monitoring_station   408251 non-null  object
 11  pm2_5                         9314 non-null    float64
 12  date                          435735 non-null  object
dtypes: float64(5), object(8)
memory usage: 43.2+ MB

Missing values:
```

	Missing Values
stn_code	144077
sampling_date	3
state	0
location	3
agency	149481
type	5393
so2	34646
no2	16233
rspm	40222
spm	237387
location_monitoring_station	27491
pm2_5	426428
date	7

```
Summary statistics:
```

	count	unique	top	freq	mean	std	min	25%	50%	75%	max
stn_code	291665	745	193	1428	NaN	NaN	NaN	NaN	NaN	NaN	NaN
sampling_date	435739	5485	19-03-15	253	NaN	NaN	NaN	NaN	NaN	NaN	NaN
state	435742	37	Maharashtra	60384	NaN	NaN	NaN	NaN	NaN	NaN	NaN
location	435739	304	Guwahati	9984	NaN	NaN	NaN	NaN	NaN	NaN	NaN
agency	286261	64	Maharashtra State Pollution Control Board	27857	NaN	NaN	NaN	NaN	NaN	NaN	NaN
type	430349	10	Residential, Rural and other Areas	179014	NaN	NaN	NaN	NaN	NaN	NaN	NaN
so2	401096.0	NaN	NaN	NaN	10.829414	11.177187	0.0	5.0	8.0	13.7	909.0
no2	419509.0	NaN	NaN	NaN	25.809623	18.503086	0.0	14.0	22.0	32.2	876.0
rspm	395520.0	NaN	NaN	NaN	108.832784	74.87243	0.0	56.0	90.0	142.0	6307.033333
spm	198355.0	NaN	NaN	NaN	220.78348	151.395457	0.0	111.0	187.0	296.0	3380.0
location_monitoring_station	408251	991	Regional Office	6261	NaN	NaN	NaN	NaN	NaN	NaN	NaN
pm2_5	9314.0	NaN	NaN	NaN	40.791467	30.832525	3.0	24.0	32.0	46.0	504.0
date	435735	5067	2015-03-19	253	NaN	NaN	NaN	NaN	NaN	NaN	NaN

3. Identify Relevant Variables

In [4]:
```
df["date"] = pd.to_datetime(df["date"], errors="coerce")
pollutant_cols = ["so2", "no2", "rspm", "spm", "pm2_5"]
for col in pollutant_cols:
    df[col] = pd.to_numeric(df[col], errors="coerce")

print("Date variable: date")
print("Pollutant variables:", pollutant_cols)
print("AQI variable: Not available in this CSV")
print("Available date range:", df["date"].min(), "to", df["date"].max())
```

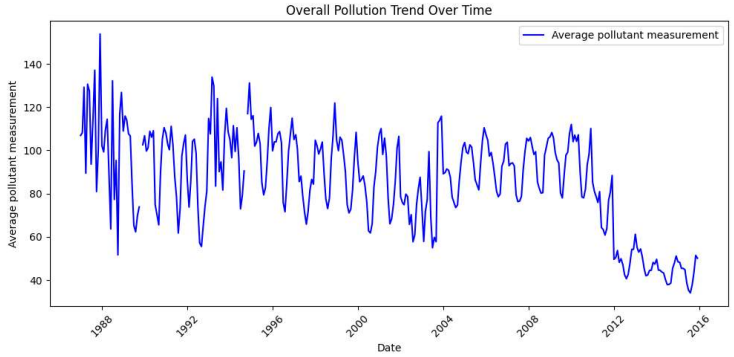
```
Date variable: date
Pollutant variables: ['so2', 'no2', 'rspm', 'spm', 'pm2_5']
AQI variable: Not available in this CSV
Available date range: 1987-01-01 00:00:00 to 2015-12-31 00:00:00
```

4. Line Plot: Overall AQI Trend Over Time

The supplied dataset does not contain an AQI column. This plot therefore uses the available pollutant measurements to show the trend in pollution levels over time; it is not an official AQI plot.

In [5]:
```
# Find the monthly average of the available pollutant readings.
monthly_pollutants = df.set_index("date")[pollutant_cols].resample("MS").mean()
monthly_pollutants["Average of available pollutant measurements"] = monthly_pollutants.mean(axis=1)

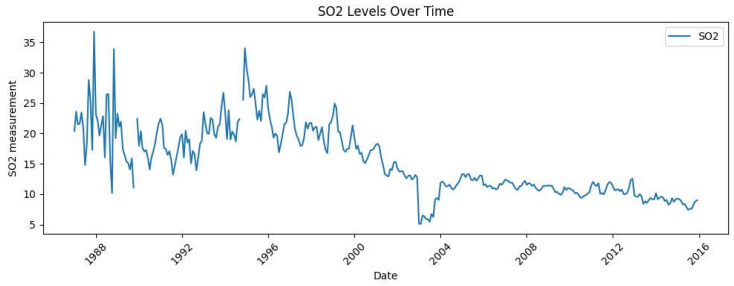
plt.figure(figsize=(10, 5))
plt.plot(monthly_pollutants.index,
         monthly_pollutants["Average of available pollutant measurements"],
         color="blue", label="Average pollutant measurement")
plt.title("Overall Pollution Trend Over Time")
plt.xlabel("Date")
plt.ylabel("Average pollutant measurement")
plt.legend()
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

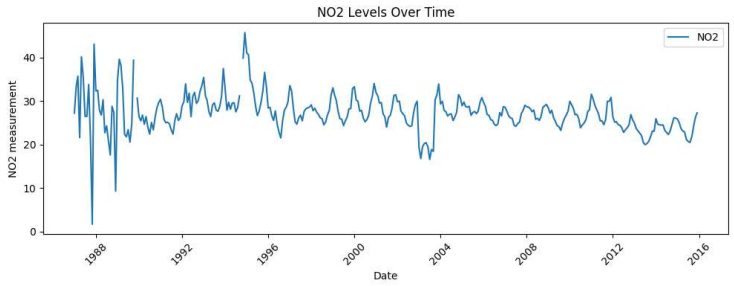


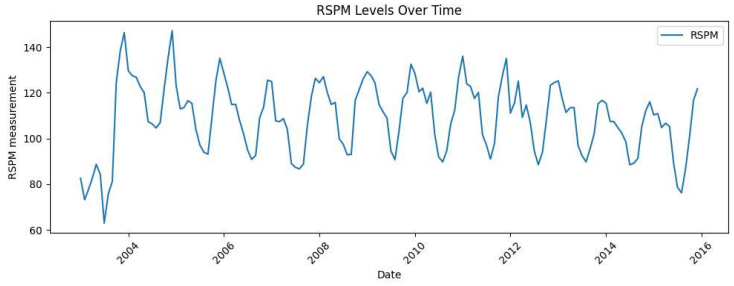
5. Line Plots: Individual Pollutant Levels Over Time

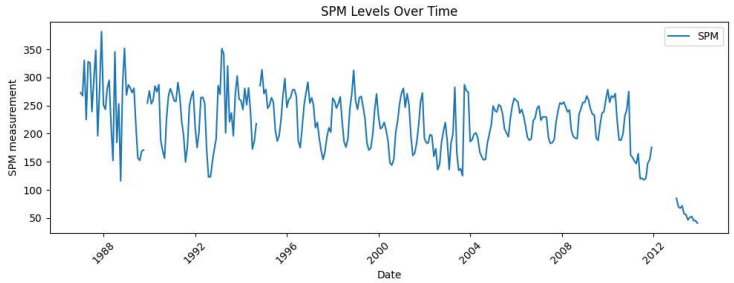
In [6]:
```
monthly_pollutants = df.set_index("date")[pollutant_cols].resample("MS").mean()

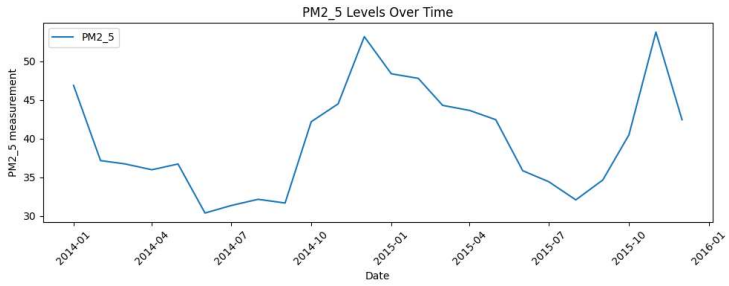
for col in pollutant_cols:
    plt.figure(figsize=(10, 4))
    plt.plot(monthly_pollutants.index, monthly_pollutants[col],
             label=col.upper(), linewidth=1.5)
    plt.title(f"{col.upper()} Levels Over Time")
    plt.xlabel("Date")
    plt.ylabel(f"{col.upper()} measurement")
    plt.legend()
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()
```







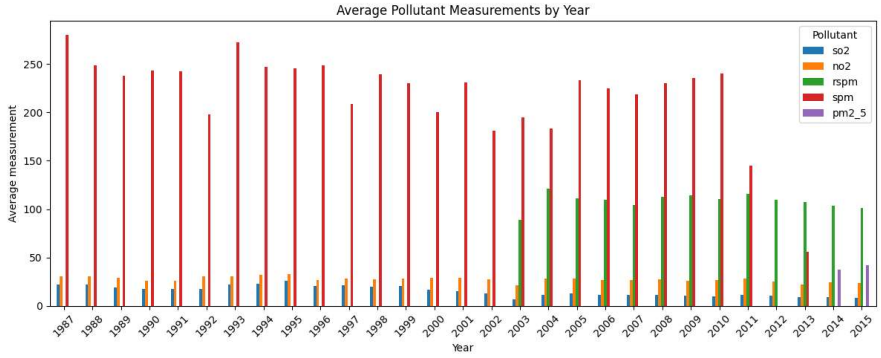




6. Bar Plot: Compare Pollution Measurements Across Years

In [7]:
```
yearly_pollutants = df.set_index("date")[pollutant_cols].resample("YS").mean()

yearly_pollutants.plot(kind="bar", figsize=(12, 5))
plt.title("Average Pollutant Measurements by Year")
plt.xlabel("Year")
plt.ylabel("Average measurement")
plt.legend(title="Pollutant")
plt.xticks(
    ticks=range(len(yearly_pollutants.index)),
    labels=yearly_pollutants.index.year,
    rotation=45
)
plt.tight_layout()
plt.show()
```



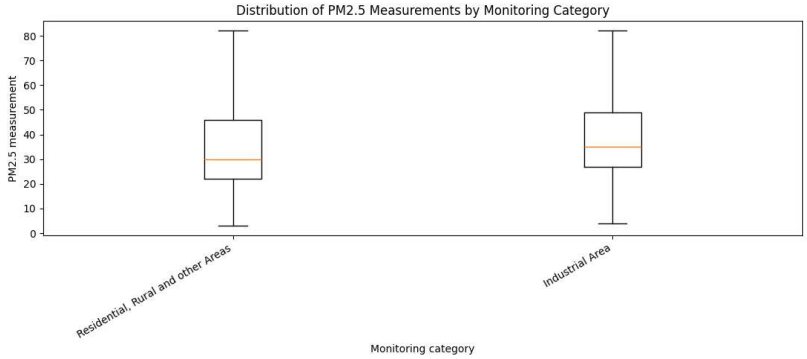
7. Box Plot: Distribution of Pollutant Measurements by Category

In [8]:
```
# Compare PM2.5 readings for the most common monitoring categories.
categories = df["type"].dropna().value_counts().head(5).index
box_data = []
box_labels = []

for category in categories:
    values = df.loc[df["type"] == category, "pm2_5"].dropna()
    if not values.empty:
        box_data.append(values.sample(n=min(5000, len(values)), random_state=42).values)
        box_labels.append(str(category))

plt.figure(figsize=(11, 5))
plt.boxplot(box_data, labels=box_labels, showfliers=False)
plt.title("Distribution of PM2.5 Measurements by Monitoring Category")
plt.xlabel("Monitoring category")
plt.ylabel("PM2.5 measurement")
plt.xticks(rotation=30, ha="right")
plt.tight_layout()
plt.show()
```

```
C:\Users\NRBor\AppData\Local\Temp\ipykernel_18816\411131139.py:13: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.
  plt.boxplot(box_data, labels=box_labels, showfliers=False)
```



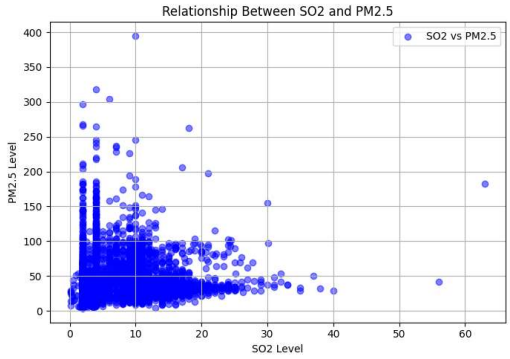
8. Scatter Plots: Relationship Between Pollutant Measurements

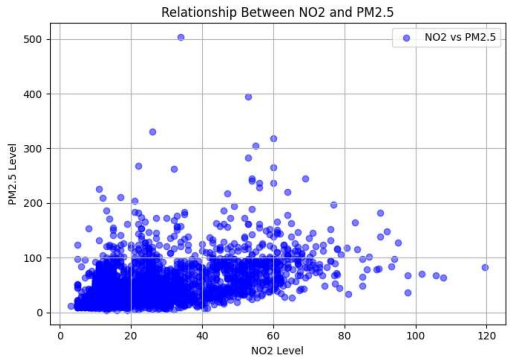
In [9]:
```
# Compare pollutant levels using scatter plots
for col in ["so2", "no2", "rspm", "spm"]:
    scatter_data = df[[col, "pm2_5"]].dropna()

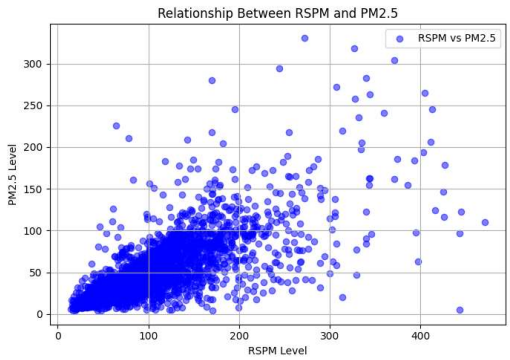
    if not scatter_data.empty:
        if len(scatter_data) > 5000:
            scatter_data = scatter_data.sample(
                n=5000, random_state=42
            )

        plt.figure(figsize=(7, 5))
        plt.scatter(
            scatter_data[col],
            scatter_data["pm2_5"],
            color="blue",
            alpha=0.5,
            label=f"{col.upper()} vs PM2.5"
        )

        plt.title(f"Relationship Between {col.upper()} and PM2.5")
        plt.xlabel(f"{col.upper()} Level")
        plt.ylabel("PM2.5 Level")
        plt.legend()
        plt.grid(True)
        plt.tight_layout()
        plt.show()
    else:
        print(
            f"No common readings available for "
            f"{col.upper()} and PM2.5."
        )
```








```
No common readings available for SPM and PM2.5.
```

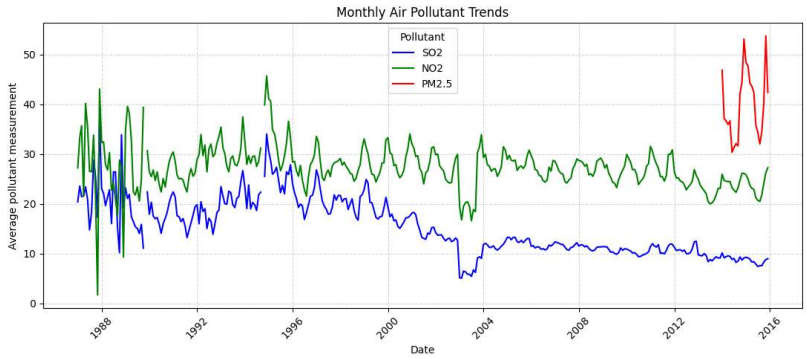
9. Customize the Visualizations

In this final task, the graph is customized with a clear title, axis labels, a legend, grid lines, and colors. These details make the chart easier to understand.

In [10]:
```
# Create a clear and easy-to-read chart with a title, labels, legend, grid, and color.
monthly = df.set_index("date")[["so2", "no2", "rspm", "spm", "pm2_5"]].resample("MS").mean()

plt.figure(figsize=(11, 5))
plt.plot(monthly.index, monthly["so2"], color="blue", label="SO2")
plt.plot(monthly.index, monthly["no2"], color="green", label="NO2")
plt.plot(monthly.index, monthly["pm2_5"], color="red", label="PM2.5")

plt.title("Monthly Air Pollutant Trends")
plt.xlabel("Date")
plt.ylabel("Average pollutant measurement")
plt.legend(title="Pollutant")
plt.grid(True, linestyle="--", alpha=0.5)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```


Monthly Air Pollutant Trends
Pollutant
SO2
NO2
PM2.5
Average pollutant measurement
0
10
20
30
40
50
1988
1992
1996
2000
2004
2008
2012
2016
Date

In [ ]: